

9.

Implementare i generics

Marco Faella

Dip. Ing. Elettrica e Tecnologie dell'Informazione
Università di Napoli “Federico II”

Corso di Linguaggi di Programmazione II

Parametri di tipo in altri linguaggi

- Il primo linguaggio a supportare parametri di tipo è stato ML (Meta-Language) nel 1973
- Parametri di tipo = *polimorfismo parametrico*
- **Java** supporta i generics dal 2005 (Java 5)
- Il **C++** ha un meccanismo simile alla programmazione generica, chiamato *template*, a partire dal 1990
- Il **C#** supporta i generics dal 2005 (C# 2.0)
- Ciascuno di questi linguaggi implementa i generics in modo diverso!

- La sintassi per le classi e i metodi template in C++ è simile a quella Java, ma l'implementazione è molto diversa:

```
template<typename T1, typename T2>
struct pair {
    T1 first;
    T2 second;

    pair(const T1& a, const T2& b) : first(a), second(b) { }
    ...
};
```

- Quando il **compilatore** C++ trova un riferimento ad una versione concreta di un template, come `pair<string,employee>`, esso **istanzia** una nuova copia della struct `pair`, con `string` al posto di `T1` ed `employee` al posto di `T2`
- Questo approccio si chiama *reificazione a tempo di compilazione*

Riassumendo:

- C++
 - **reificazione** a tempo di compilazione
 - il compilatore crea una versione specifica del codice per ogni parametro attuale di tipo
- C#
 - **reificazione** a tempo di esecuzione
 - come in C++, ma a tempo di esecuzione (*on demand*), da parte della macchina virtuale
- Java
 - **cancellazione** (*erasure*)
 - il compilatore usa i generics per fare un type checking più accurato, poi *scarta* l'informazione

- Approfondiamo il meccanismo implementativo scelto da Java e le limitazioni che questo meccanismo comporta sull'uso dei parametri di tipo
- Il meccanismo che supporta la programmazione generica prende il nome di **cancellazione** (in inglese, **erasure**)
- Il principio di funzionamento è il seguente:
 - 1) I parametri di tipo vengono usati dal compilatore per effettuare i dovuti controlli di tipo (*type checking*)
 - 2) Poi, tutti i parametri di tipo, formali e attuali, vengono **rimossi** (cancellati, appunto) e sostituiti da `Object`, oppure dal *primo limite superiore* del parametro in questione, se presente
 - 3) Il parametro di tipo jolly viene semplicemente rimosso
 - 4) In conseguenza della cancellazione, vengono inseriti degli opportuni cast, per ripristinare la coerenza tra tipi
- Il **bytecode** di una classe o metodo parametrico mantiene solo le informazioni minime sui generics necessarie a svolgere i controlli di tipo sugli utenti di quella classe/metodo (segue esempio)
- Ovvero, in fase di esecuzione è come se i generics fossero scomparsi
- In breve: i generics sono interamente gestiti a **tempo di compilazione**, senza alcun impatto a runtime

Limitazioni dei generics

Come conseguenza dell'erasure:

I parametri attuali di tipo non possono produrre effetti a runtime

A causa di questa limitazione, il linguaggio limita *a tempo di compilazione* quello che si può fare con i parametri formali di tipo

Non è possibile utilizzare un parametro formale di tipo per istanziare oggetti

`new T()` *(errore di compilazione)*

- Se fosse consentita, quest'istruzione **produrrebbe effetti a runtime** dipendenti dal parametro attuale che sostituisce T, cosa impossibile a causa dell'erasure
- In altri termini, a runtime tale istruzione diventerebbe "`new Object()`", che sicuramente non è quello che il programmatore intendeva ottenere
- D'altronde, è possibile utilizzare un parametro di tipo per istanziare una classe concreta, come in:

`new LinkedList<T>()`

- Infatti, quest'istruzione non produce effetti a runtime dipendenti dal parametro attuale che sostituisce T

Non è possibile utilizzare il jolly per istanziare oggetti

(questa regola è indipendente dall'erasure)

- Il parametro di tipo jolly non può essere utilizzato neanche per istanziare una classe concreta

`new LinkedList<?>()` *(errore di compilazione)*

- Quest'istruzione corrisponderebbe alla richiesta di creare una lista di tipo sconosciuto
- Ricordiamo che il parametro di tipo jolly **serve per avere riferimenti in grado di puntare a diverse versioni di una classe parametrica**
- Se il nostro intento è di creare una lista che possa contenere qualsiasi oggetto, il tipo giusto è `LinkedList<Object>`

Non è possibile istanziare un array di tipo parametrico

`new T[10]` (*errore di compilazione*)

- Gli array ricordano il tipo con il quale sono stati creati
- Se fosse consentita, quest'istruzione produrrebbe effetti a runtime dipendenti dal parametro attuale che sostituisce T
- Una possibile soluzione consiste nell'istanziare una lista di tipo T, invece di un array
- Invece, è possibile dichiarare un riferimento di tipo array di tipo parametrico

`T[] a;`

- Questo costrutto è utile, ad esempio, come parametro formale di un metodo, per accettare array di qualsiasi tipo ed associare il tipo dell'array a quello di altri parametri, oppure al tipo di ritorno

- I parametri di tipo presentano una serie di problemi legati all'overloading
- Per prima cosa:

Non è possibile usare i parametri di tipo per distinguere due versioni di un metodo

- Consideriamo la classe `Pair<S,T>`, che rappresenta una coppia di elementi di due tipi diversi
- Si potrebbe essere tentati di realizzarla secondo il seguente schema:

```
public class Pair<S, T> {  
    private S first;  
    private T second;  
    public void setValue(S x) { first = x; }  
    public void setValue(T x) { second = x; }    (errore di compilazione)  
    ...  
}
```

- Questa soluzione non funziona perché entrambi i parametri di tipo, dopo il type checking, diventeranno `Object`, e quindi l'overloading diventerà non valido

Un caso speciale di overloading

- Consideriamo una variante di Pair in cui la seconda componente deve essere sottotipo di Number:

```
public class OddPair<S, T extends Number> {  
    public void setValue(S s) { S.o.println("Versione S"); }  
    public void setValue(T t) { S.o.println("Versione T"); }  
}
```

- Questo overloading è lecito, perché l'erasure di S (cioè, Object) è diversa dall'erasure di T (che è il primo limite superiore di T, ovvero Number)
- Ora, consideriamo questo uso:

```
OddPair<Integer,Integer> p = new OddPair<>();
```

- La riga precedente è lecita, anche se rende l'overloading ambiguo!
- Tuttavia, qualsiasi tentativo di invocare p.setValue produrrà un errore di compilazione:

```
p.setValue(Integer.valueOf(42));
```

errore di compilazione: ambiguità

```
p.setValue(new Object());
```

errore di compilazione: nessuna firma candidata

Nota: inoltre, non è possibile estendere OddPair<Integer,Integer>

Un caso speciale di overloading

- Consideriamo una variante di Pair in cui la seconda componente deve essere sottotipo di Number:

```
public class OddPair<S, T extends Number> {  
    public void setValue(S s) { S.o.println("Versione S"); }  
    public void setValue(T t) { S.o.println("Versione T"); }  
}
```

- Questo overloading è lecito, perché l'erasure di S (cioè, Object) è diversa dall'erasure di T (che è il primo limite superiore di T, ovvero Number)
- Ora, consideriamo questo uso:

```
OddPair<Integer,Integer> p = new OddPair<>();
```

- La riga precedente è lecita, anche se rende l'overloading ambiguo!
- Tuttavia, qualsiasi tentativo di invocare p.setValue produrrà un errore di compilazione:

```
p.setValue(Integer.valueOf(42));
```

errore di compilazione: *ambiguità*

```
p.setValue(new Object());
```

errore di compilazione: *nessuna firma candidata*

Nota: inoltre, non è possibile estendere OddPair<Integer,Integer>

- Se invece usiamo la **versione grezza** della classe, il compilatore ignorerà i generics e userà solo l'erasure dei tipi:

```
OddPair q = new OddPair();
```

warning: uso di classe grezza

```
q.setValue(Integer.valueOf(42));
```

stampa "Versione T"

```
q.setValue("ciao");
```

stampa "Versione S"

Un caso speciale di overloading

- L'esempio precedente mostra che il bytecode conserva informazioni sufficienti per effettuare il type-checking e la risoluzione dell'overloading tenendo conto dei parametri attuali di tipo
- Questa non è una violazione della regola "*i parametri attuali di tipo non possono produrre effetti a runtime*", perché type-checking e risoluzione dell'overloading avvengono a **tempo di compilazione**
- Verifichiamo le informazioni contenute nel bytecode col comando javap (java print):

```
> javap -p -s -c OddPair.class
```

```
Compiled from "OddPair.java"
```

```
class OddPair<S, T extends java.lang.Number> {  
    [...]  
    void setValue(S);  
        descriptor: (Ljava/lang/Object;)V  
        Code: [...]  
  
    void setValue(T);  
        descriptor: (Ljava/lang/Number;)V  
        Code: [...]  
}
```

Parametri di tipo formali della classe.
Usati per fare il type-checking dei client di OddPair.
Non si può permettere OddPair<Object,String>!

Parametro formale S, prima dell'erasure.
Usato per fare il type-checking dei client.

Parametro formale Object, dopo l'erasure.
Usato a runtime e per compilare i client della classe grezza.

Il compilatore vede S.
La VM vede Object.

Dopo l'erasure, un metodo parametrico potrebbe andare in conflitto con uno non parametrico

- Ad esempio, non possiamo avere i seguenti metodi nella stessa classe:

```
public <T> void f(T t)      { ... }  
public      void f(Object o) { ... } (errore di compilazione)
```

- Invece, i seguenti metodi possono coesistere:

```
public <T> void f(T t)      { ... }  
public      void f(String s) { ... }
```

- La seconda versione specializza la prima e verrà invocata sulle stringhe
 - Anche in caso di un'invocazione del tipo `x.<String>f("ciao")!`

Non è possibile utilizzare un parametro di tipo
per selezionare una determinata versione di un metodo in overloading

- Ad esempio, consideriamo i seguenti metodi:

```
public void f(String s) { ... }  
public void f(Object o) { ... }
```

```
public <T> void g(T x) { f(x); }
```

- Indipendentemente dal valore assunto dal parametro di tipo T, il metodo **g chiamerà sempre f(Object)**
 - in particolare, anche se g sarà chiamato con T=String
- Infatti, la risoluzione dell'overloading avviene a tempo di compilazione, quando ancora non si conosce il valore che T assumerà
 - il compilatore non può che assumere T=Object

I parametri di tipo non vanno usati per effettuare conversioni esplicite (**cast**)

- In particolare, abbiamo:

`(T) x;` *(warning)*

a run-time diventa un cast verso `Object` (o verso il primo limite superiore di `T`, se presente)

- Qualunque cast verso un tipo parametrico (un parametro di tipo oppure una classe o interfaccia parametrica) produce un warning in compilazione:

`(LinkedList<String>) x` *(warning)*

- Se è necessario effettuare un cast (es., metodo `equals`), lo si può fare usando il parametro jolly:

`(LinkedList<?>) x`

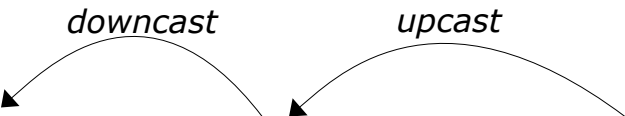
1) `(List<String>) new Object()`

Warning, poi eccezione a run-time perché il tipo effettivo (`Object`) non è sottotipo di `List`

2) `(List<String>) new LinkedList<Integer>()`

Errore di compilazione perché `LinkedList<Integer>` non è sottotipo di `List<String>` (né viceversa)

3) `(List<String>) (List<?>) new LinkedList<Integer>()`



The diagram illustrates the casting process in the third example. It shows two types: `List<String>` on the left and `List<?>` in the middle. A curved arrow labeled "downcast" points from `List<?>` to `List<String>`. Another curved arrow labeled "upcast" points from `new LinkedList<Integer>()` to `List<?>`.

Warning, nessuna eccezione a run-time;

con questa sequenza di conversioni possiamo inserire una stringa in una lista di interi (peggio per noi...)

**Non si può applicare instanceof a un parametro formale di tipo
o a una classe parametrica**

Esempio:

`x instanceof T` *(errore di compilazione)*

A run-time diventerebbe "x instanceof Object"

Quindi, il risultato sarebbe sempre true (tranne che per null)

Non si può applicare instanceof a un parametro di tipo o a una classe parametrica

Inoltre:

```
x instanceof List<T>           (errore di compilazione)  
x instanceof List<String>      (errore di compilazione)
```

La JVM non può controllare se x è sottotipo di List<T> o List<String>, perché a runtime x sarà semplicemente una List (o una LinkedList, ArrayList, etc.)

Questo contesto è uno dei pochi casi in cui è opportuno usare la versione grezza:

```
x instanceof List
```

oppure:

```
x instanceof List<?>
```

Vantaggi della reificazione (C#, C++):

- Espressività: si può fare con un parametro di tipo tutto quello che si può fare con un tipo concreto

Vantaggi della cancellazione (Java):

- Evita il *code bloating* (ripetizione, nell'eseguibile o in memoria, di codice simile)
- Supporta la compilazione separata

- Nella lezione precedente, abbiamo esaminato il seguente metodo della classe `java.util.Collections`:

```
public static <T extends Object & Comparable<? super T>> T min(Collection<? extends T> l)
```

- A questo punto, è possibile analizzare in dettaglio questa firma
- Per individuare l'elemento minimo di una collezione, il metodo `min` ha bisogno che gli elementi della collezione siano mutuamente confrontabili tramite l'interfaccia `Comparable`
- Quindi, il parametro di tipo `T`, che rappresenta il tipo degli elementi della collezione, ha come limite superiore `Comparable<? super T>`
- Inoltre, prima che esistessero i tipi parametrici in Java, questo metodo era già presente, con la firma

```
public static Object min(Collection l)
```

- Il limite superiore `<T extends Object & ...>` fa sì che, dopo la cancellazione, la nuova firma coincida con la vecchia, a vantaggio della compatibilità con il codice pre-esistente
- Infine, il tipo del parametro `Collection<? extends T>` offre la garanzia che la collezione passata al metodo `min` non sarà modificata

Confronto riflessione vs
generics

Consideriamo una classe per coppie di oggetti dello stesso tipo

Con *generics*:

```
public class Pair<S> {  
    private S first, second;  
  
    public Pair(S a, S b) {  
        first = a;  
        second = b;  
    }  
    public void setFirst(S a) { first = a; }  
    public S getFirst() { return first; }  
  
    @Override  
    public boolean equals(Object other) {  
        if (!(other instanceof Pair))  
            return false;  
        Pair<?> p = (Pair) other;  
        return first.equals(p.first) &&  
            second.equals(p.second);  
    }  
}
```

Siamo costretti dall'erasure
a usare la classe grezza

Non possiamo controllare
se other è una coppia dello stesso tipo ☹️

Ora proviamo a simulare i generics con la **riflessione**:

```
public class Pair {  
    private Object first, second;  
    private final Class<?> type;  
  
    public Pair(Class<?> c, Object a, Object b)  
    {  
        if (!c.isInstance(a) || !c.isInstance(b))  
            throw new IllegalArgumentException();  
        type = c;  
        first = a;  
        second = b;  
    }  
    public void setFirst(Object a) {  
        if (!type.isInstance(a))  
            throw new IllegalArgumentException();  
        first = a;  
    }  
    public Object getFirst() { return first; }  
}
```

Controlliamo a runtime
quello che i generics controllano
in fase di compilazione ☹

continua...

Ora proviamo a simulare i generics con la **riflessione**:

```
public class Pair {  
    private Object first, second;  
    private final Class<?> type;  
  
    ...  
  
    @Override  
    public boolean equals(Object other) {  
        if (!(other instanceof Pair))  
            return false;  
        Pair p = (Pair) other;  
        if (p.type != type)  
            return false;  
        return first.equals(p.first) &&  
            second.equals(p.second);  
    }  
}
```

Possiamo controllare
il tipo effettivo di un'altra coppia

Attenzione: così una coppia di Manager risulta
diversa da una coppia di Employee, anche se
contengono gli stessi oggetti (due Manager)

Possiamo anche **combinare i due approcci**

Generics + riflessione:

```
public class Pair<S> {  
    private S first, second;  
    private final Class<S> type;  
  
    public Pair(Class<S> c, S a, S b)  
    {  
        type = c;  
        first = a;  
        second = b;  
    }  
    public void setFirst(S a) { first = a; }  
    public S    getFirst() { return first; }  
    ...  
}
```

Usare questa classe:

```
Pair<String> p = new Pair<>(String.class, "uno", "due");      OK
```

```
Pair<String> p = new Pair<>("pippo".getClass(), "uno", "due");  ??
```